

GRAPH ALGORITHMS

POWERING THE AI STUDY PLANNER CO-EVOLUTION



Graph Algorithms - Adaptive AI Study Planner with Human-Algorithm Co-evolution

This section gives an overview of the graph algorithms used in our project to manage subject dependencies and generate a proper study order.

Team 17 (AIE-A)

Prakeya S - (CB.SC.U4AIE25042)

Sliding Window Algorithm

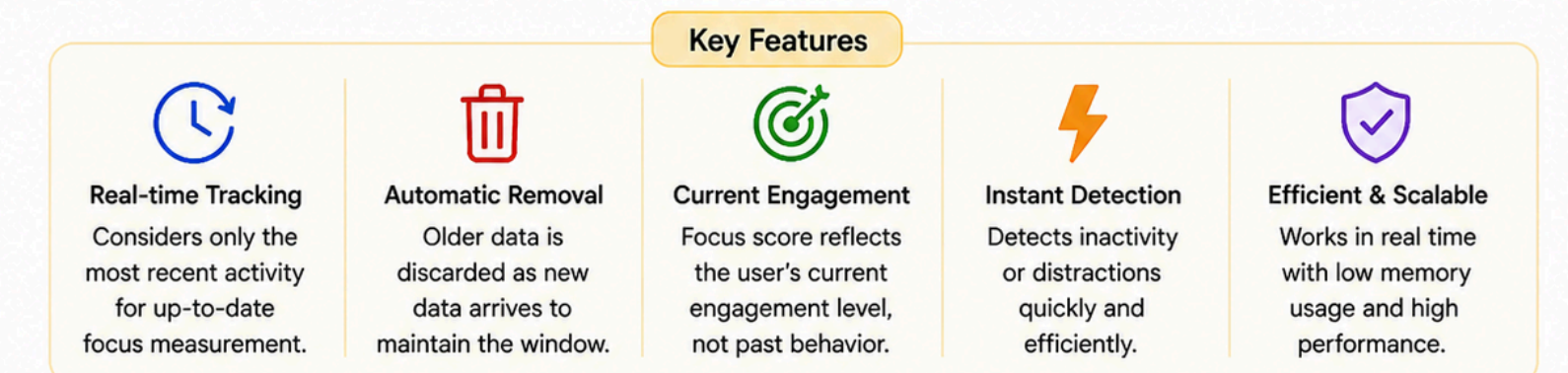
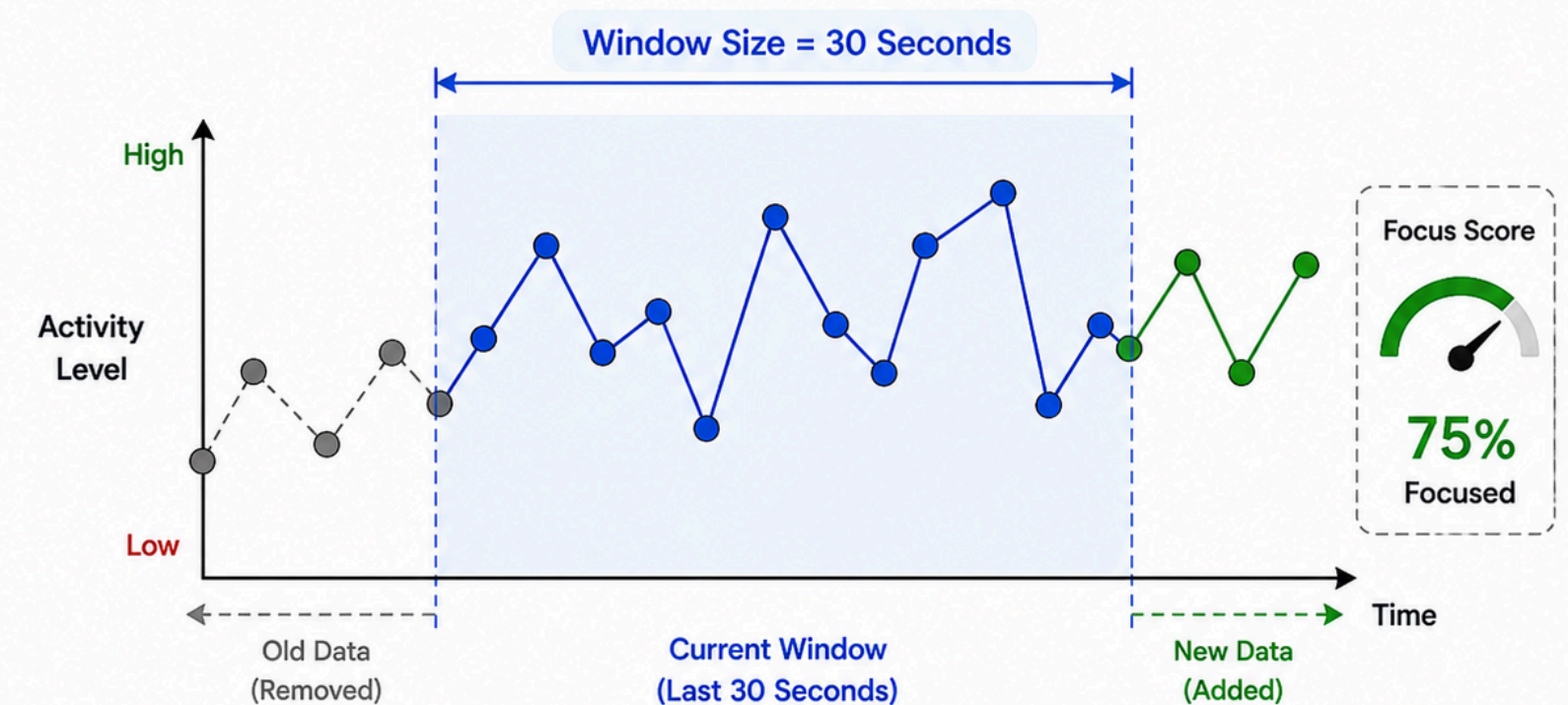
- The sliding window algorithm processes data within a fixed range that moves over time. In this project, it tracks recent user activity by keeping only current inputs and removing older data. This ensures the system reflects real-time behavior and provides accurate focus tracking.

Key Points

- Maintains a fixed time window (e.g., last 30 seconds)
- Removes outdated data as new activity arrives
- Continuously updates results based on recent data

Application in Project

- Calculates real-time focus score
- Detects inactivity and distractions
- Triggers warnings and break suggestions



DFS (Depth First Search)

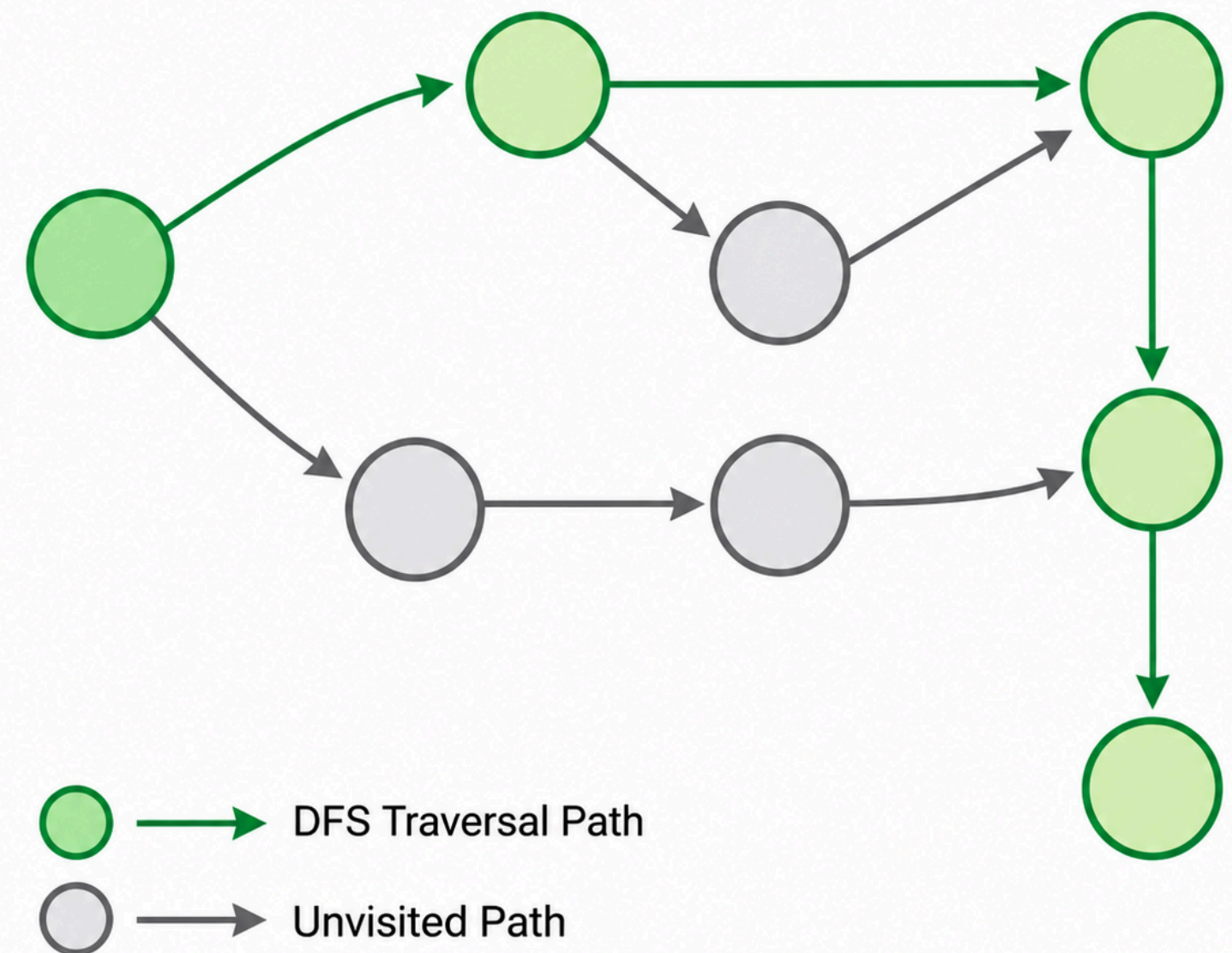
- DFS is a graph traversal algorithm that explores nodes deeply before moving to the next branch. In this project, it is used to detect cycles in the subject dependency graph. By tracking visited nodes, it ensures no circular dependencies exist. This guarantees a valid structure for learning paths.

Key Points

- Traverses graph by going deep into one path before switching
- Uses recursion or a stack to manage traversal
- Keeps track of visited nodes to avoid revisiting

Application in Project

- Detects cycles in subject dependencies
- Prevents invalid learning paths like $A \rightarrow B \rightarrow A$
- Ensures the graph structure is logically correct



Kahn's Algorithm

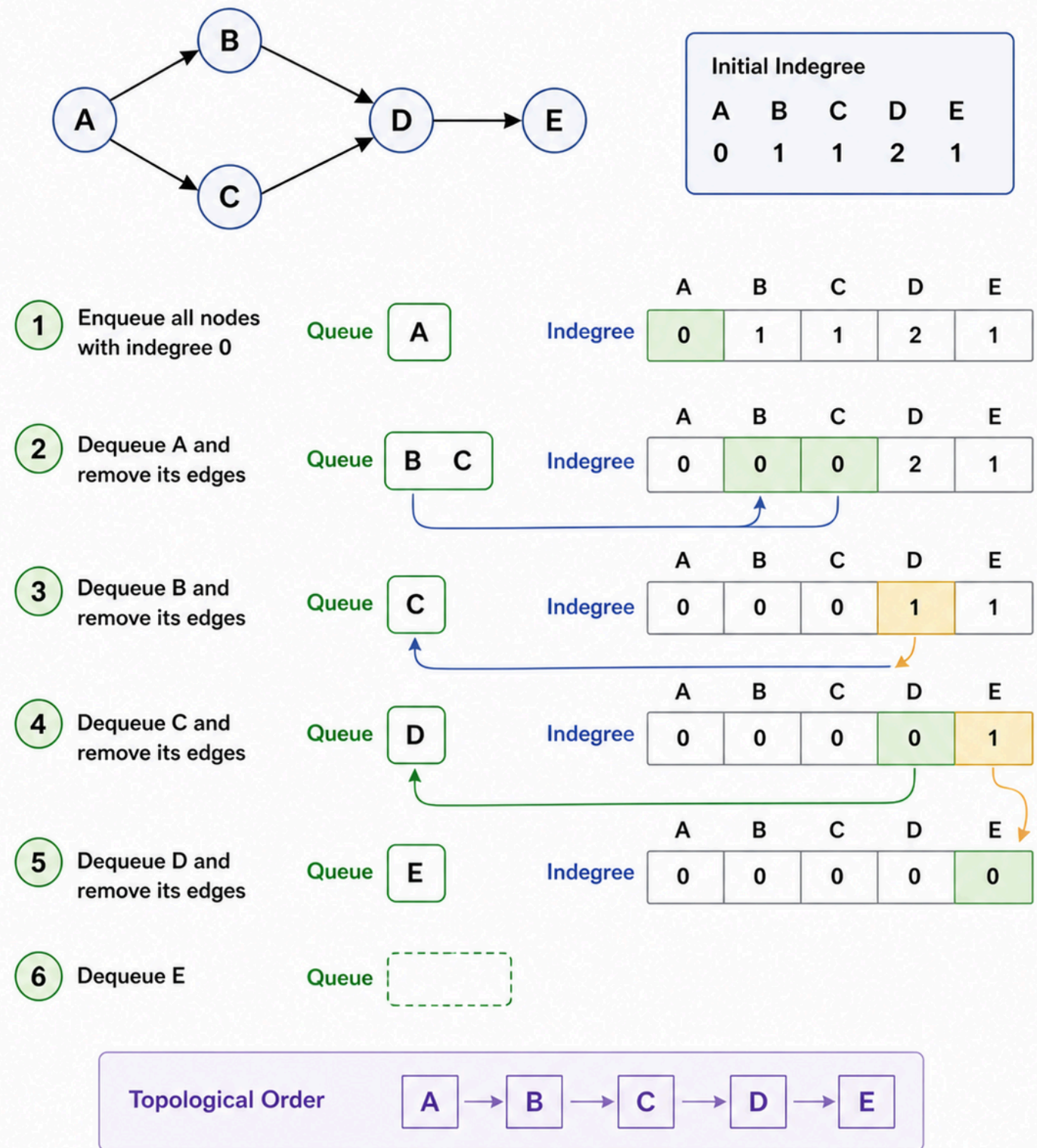
- Kahn's Algorithm is used to perform topological sorting in a directed graph. It works by repeatedly selecting nodes with no incoming edges and processing them. In this project, it generates the correct study order. This ensures prerequisites are completed before dependent subjects.

Key Points

- Uses a queue to process nodes step by step
- Always selects nodes with indegree = 0
- Removes processed nodes and updates remaining graph

Application in Project

- Generates the correct study sequence
- Ensures subjects are learned in proper order
- Processes only ready subjects (no prerequisites)



Indegree Calculation

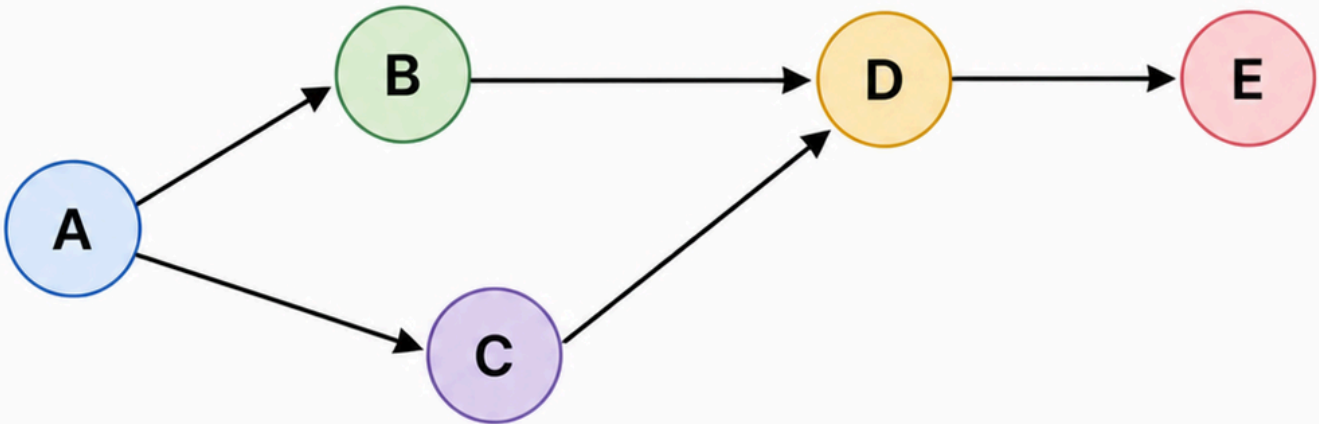
- Indegree calculation determines the number of prerequisites each subject has in the graph. It helps identify which subjects can be studied first based on having no dependencies. As subjects are completed, these values are updated to reflect the current state of the learning sequence.

Key Points

- Counts number of prerequisites for each subject
- Helps identify starting nodes in the graph
- Continuously updated as dependencies are resolved

Application in Project

- identifies subjects that can be studied immediately
- Supports Kahn's Algorithm for ordering
- Controls when subjects become available



1 Initialize indegree of all nodes to 0

A	B	C	D	E
0	0	0	0	0

2 For each edge (u → v), increase indegree of v by 1

A → B	0	1	0	0	0
A → C	0	1	1	0	0
B → D	0	1	1	1	0
C → D	0	1	1	2	0
D → E	0	1	1	2	1

3 Final indegree of all nodes

A	B	C	D	E
0	1	1	2	1

State Checking (Unlock Logic)

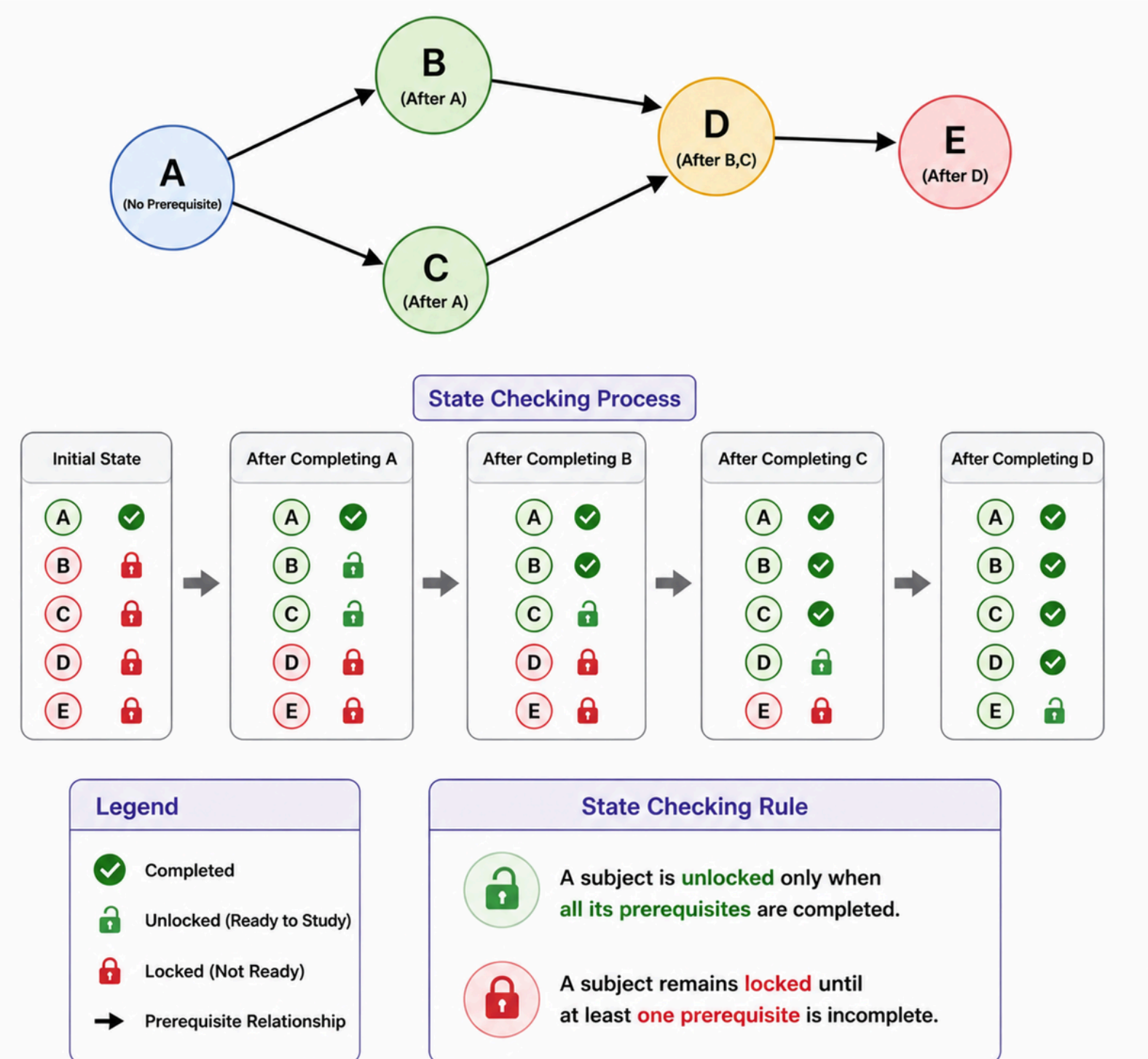
- State checking ensures that a subject becomes available only after all its prerequisites are completed. It verifies the completion status before allowing access to the next subject. This helps maintain a proper learning sequence and prevents users from skipping important topics.

Key Points

- Checks if all prerequisites are completed
- Uses a completed set for verification
- Prevents accessing locked subjects

Application in Project

- Unlocks subjects step-by-step
- Prevents skipping important topics
- Maintains correct learning progression



Priority Calculation

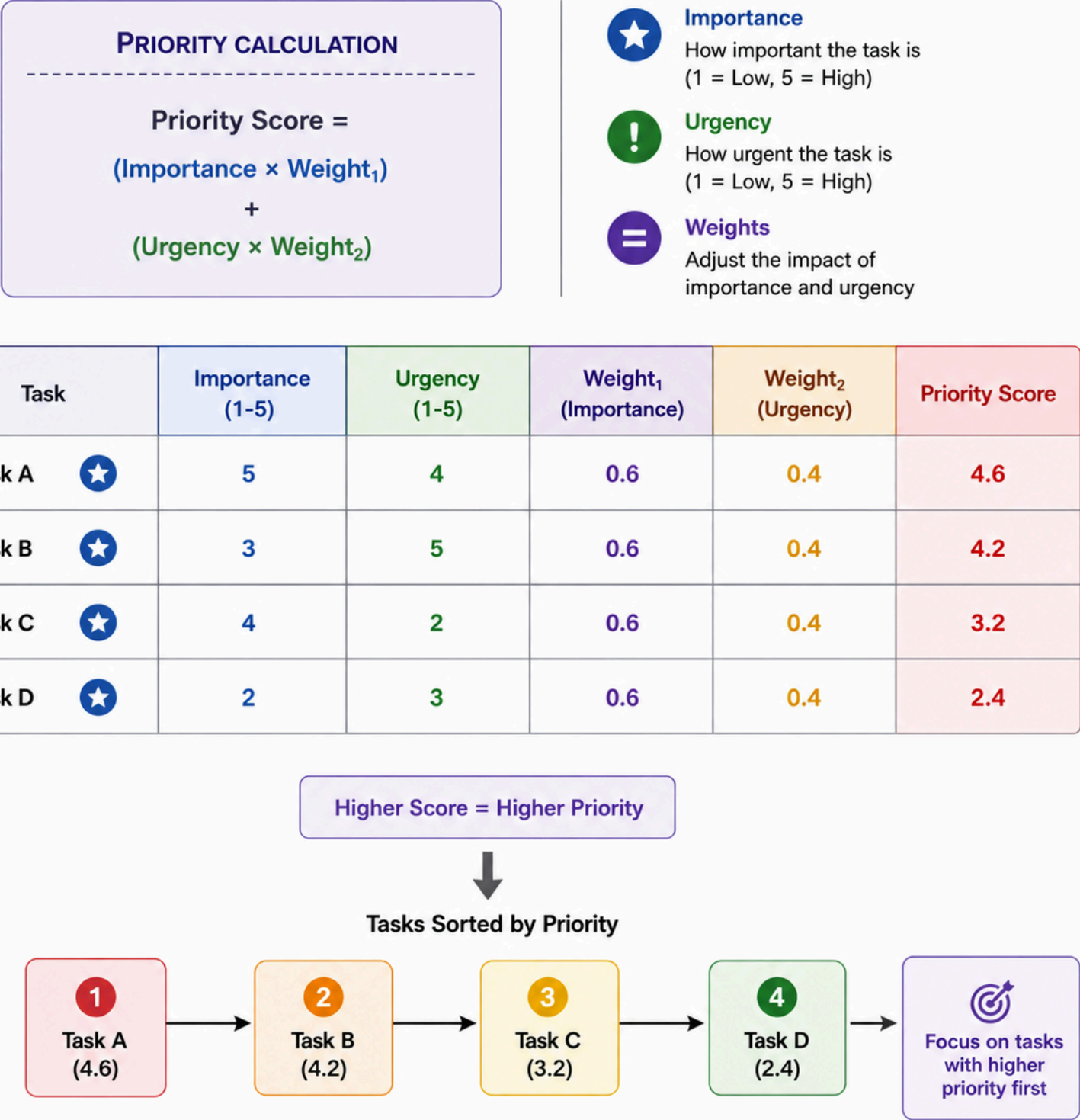
- Priority calculation assigns a score to each task based on its importance and urgency. It helps determine which task should be given more attention at a particular time. This allows the system to guide users toward completing the most critical tasks efficiently.

Key Points

- Combines importance and urgency into a single score
- Higher score means higher priority
- Updates dynamically as time changes

Application in Project

- Highlights most important tasks
- Helps users focus on urgent work
- Improves task management efficiency



Sorting / Priority Queue Logic

- Sorting or priority queue logic arranges tasks based on their priority values. It ensures that tasks with higher priority are handled before others in the list. This enables efficient and dynamic task scheduling as priorities change over time.

Key Points

- Orders tasks from highest to lowest priority
- Uses comparison logic for arrangement
- Updates order whenever priority changes

Application in Project

- Displays most important task at top
- Maintains structured task list
- Supports real-time updates

PRIORITY CALCULATION

Priority Score =
(Importance × Weight₁)
+
(Urgency × Weight₂)

- ★

Importance

How important the task is
(1 = Low, 5 = High)
- !

Urgency

How urgent the task is
(1 = Low, 5 = High)
- =

Weights

Adjust the impact of
importance and urgency

Task	Importance (1-5)	Urgency (1-5)	Weight ₁ (Importance)	Weight ₂ (Urgency)	Priority Score
Task A ★	5	4	0.6	0.4	4.6
Task B ★	3	5	0.6	0.4	4.2
Task C ★	4	2	0.6	0.4	3.2
Task D ★	2	3	0.6	0.4	2.4

